

```
import pandas as pd

# Load the dataset
df = pd.read_csv('Invistico_Airline.csv')

# Display the first 5 rows of the DataFrame
print("First 5 rows of the dataset:")
display(df.head())

# Inspect the target variable 'satisfaction'
print("\nDistribution of the 'satisfaction' target variable:")
display(df['satisfaction'].value_counts())
```

First 5 rows of the dataset:

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	location
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	0	

5 rows × 22 columns

Distribution of the 'satisfaction' target variable:

	count
satisfaction	
satisfied	71087
dissatisfied	58793

dtype: int64

```
# Convert 'satisfaction' to numerical (0 and 1)
df['satisfaction'] = df['satisfaction'].apply(lambda x: 1 if x == 'satisfied' else 0)

# Identify categorical columns (excluding 'satisfaction' which is already handled)
categorical_cols = df.select_dtypes(include=['object']).columns
print(f"Categorical columns to encode: {list(categorical_cols)}")

# Apply one-hot encoding to categorical columns
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Handle missing values in 'Arrival Delay in Minutes' by filling with the mean
# (considering the project scope, a simple imputation is chosen)
df_encoded['Arrival Delay in Minutes'].fillna(df_encoded['Arrival Delay in Minutes'].mean(), inplace=True)

# Display the first few rows of the encoded DataFrame and its info to confirm changes
print("\nFirst 5 rows of the preprocessed dataset:")
display(df_encoded.head())

print("\nInfo of the preprocessed dataset:")
df_encoded.info()
```

Categorical columns to encode: ['Customer Type', 'Type of Travel', 'Class']

First 5 rows of the preprocessed dataset:

```
/tmp/ipykernel_3100/3248114218.py:13: FutureWarning: A value is trying to be set on a copy of a DataFrame.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate result will be discarded.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'.

df_encoded['Arrival Delay in Minutes'].fillna(df_encoded['Arrival Delay in Minutes'].mean(), inplace=True)

	satisfaction	Age	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	Inflight wifi service	Inflight entertainment
0	1	65	265	0	0	0	2	2	
1	1	47	2464	0	0	0	3	0	
2	1	15	2138	0	0	0	3	2	
3	1	60	623	0	0	0	3	3	
4	1	70	354	0	0	0	3	4	

5 rows × 23 columns

Info of the preprocessed dataset:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 23 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   satisfaction                             129880 non-null  int64
1   Age                                       129880 non-null  int64
2   Flight Distance                         129880 non-null  int64
3   Seat comfort                           129880 non-null  int64
4   Departure/Arrival time convenient      129880 non-null  int64
5   Food and drink                         129880 non-null  int64
6   Gate location                         129880 non-null  int64
7   Inflight wifi service                  129880 non-null  int64
8   Inflight entertainment                 129880 non-null  int64
9   Online support                        129880 non-null  int64
10  Ease of Online booking                 129880 non-null  int64
11  On-board service                      129880 non-null  int64
12  Leg room service                     129880 non-null  int64
13  Baggage handling                     129880 non-null  int64
14  Checkin service                      129880 non-null  int64
15  Cleanliness                          129880 non-null  int64
16  Online boarding                      129880 non-null  int64
17  Departure Delay in Minutes            129880 non-null  int64
18  Arrival Delay in Minutes              129880 non-null  float64
19  Customer Type_disloyal Customer      129880 non-null  bool
20  Type of Travel_Personal Travel        129880 non-null  bool
21  Class_Eco                            129880 non-null  bool
22  Class_Eco Plus                       129880 non-null  bool
dtypes: bool(4), float64(1), int64(18)
memory usage: 19.3 MB
```

```
from sklearn.model_selection import train_test_split

# Define features (X) and target (y)
X = df_encoded.drop('satisfaction', axis=1)
y = df_encoded['satisfaction']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

print("\nDistribution of satisfaction in training set:")
display(y_train.value_counts(normalize=True))
print("\nDistribution of satisfaction in testing set:")
display(y_test.value_counts(normalize=True))
```

Shape of X_train: (90916, 22)
Shape of X_test: (38964, 22)
Shape of y_train: (90916,)
Shape of y_test: (38964,)

Distribution of satisfaction in training set:

proportion	
satisfaction	
1	0.547329
0	0.452671

dtype: float64

Distribution of satisfaction in testing set:

proportion	
satisfaction	
1	0.547326
0	0.452674

dtype: float64

```
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model
# Using solver='liblinear' for smaller datasets and for 'l1' regularization, if needed
# For larger datasets, 'lbfgs' or 'saga' might be more appropriate.
# max_iter is increased to ensure convergence.
model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)

# Train the model
print("Training Logistic Regression model...")
model.fit(X_train, y_train)
print("Model training complete.")

# Make predictions on the test set
y_pred = model.predict(X_test)

# Predict probabilities for the 'satisfied' class (class 1)
y_pred_proba = model.predict_proba(X_test)[:, 1]

print("\nFirst 10 actual vs predicted values on test set:")
display(pd.DataFrame({'Actual': y_test.head(10), 'Predicted': y_pred[:10]}))
print("\nFirst 10 predicted probabilities for satisfaction:")
display(y_pred_proba[:10])
```

Training Logistic Regression model...
Model training complete.

First 10 actual vs predicted values on test set:

	Actual	Predicted	
86347	1	1	
19147	0	0	
52126	0	0	
90451	1	1	
70261	0	0	
15029	0	0	
54234	0	0	
114164	1	1	
58506	0	0	
73584	0	0	

First 10 predicted probabilities for satisfaction:
array([0.93646488, 0.12573652, 0.04401803, 0.94118155, 0.11468385,
0.11247274, 0.28293219, 0.97951913, 0.19679916, 0.08422165])

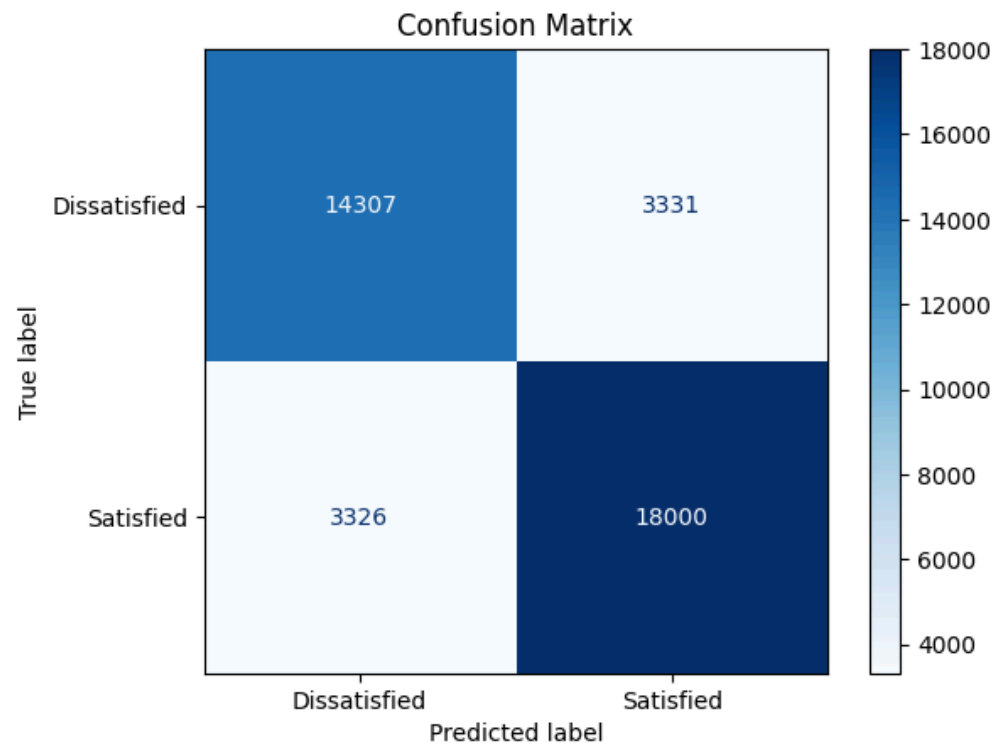
```
from sklearn.metrics import confusion_matrix, precision_score, recall_score, accuracy_score, Confu
import matplotlib.pyplot as plt
```

```
# Generate Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Display Confusion Matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Dissatisfied', 'Satisfied'])
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.show()

# Calculate Precision and Recall
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print(f"\nModel Accuracy: {accuracy:.4f}")
print(f"Model Precision: {precision:.4f}")
print(f"Model Recall: {recall:.4f}")
```



Model Accuracy: 0.8291
Model Precision: 0.8438
Model Recall: 0.8440

```
# Get feature names and their corresponding coefficients
feature_names = X.columns
coefficients = model.coef_[0]

# Create a DataFrame for better readability
coef_df = pd.DataFrame({'Feature': feature_names, 'Coefficient': coefficients})

# Sort by the absolute value of coefficients to see the most impactful features
coef_df['Abs_Coefficient'] = abs(coef_df['Coefficient'])
coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending=False)

print("Top 10 features by absolute coefficient value (most impactful):")
display(coef_df.head(10))

print("\nAll features and their coefficients:")
display(coef_df.sort_values(by='Coefficient', ascending=False))
```

Top 10 features by absolute coefficient value (most impactful):				
	Feature	Coefficient	Abs_Coefficient	
18	Customer Type_disloyal Customer	-1.878154	1.878154	
21	Class_Eco Plus	-0.787932	0.787932	
19	Type of Travel_Personal Travel	-0.739723	0.739723	
20	Class_Eco	-0.729280	0.729280	
7	Inflight entertainment	0.719635	0.719635	
10	On-board service	0.308925	0.308925	
13	Checkin service	0.283897	0.283897	
2	Seat comfort	0.282618	0.282618	
9	Ease of Online booking	0.253266	0.253266	
11	Leg room service	0.239172	0.239172	
All features and their coefficients:				
	Feature	Coefficient	Abs_Coefficient	
7	Inflight entertainment	0.719635	0.719635	
10	On-board service	0.308925	0.308925	
13	Checkin service	0.283897	0.283897	
2	Seat comfort	0.282618	0.282618	
9	Ease of Online booking	0.253266	0.253266	
11	Leg room service	0.239172	0.239172	
15	Online boarding	0.141383	0.141383	
5	Gate location	0.126438	0.126438	
8	Online support	0.109265	0.109265	
12	Baggage handling	0.092364	0.092364	
14	Cleanliness	0.048885	0.048885	
16	Departure Delay in Minutes	0.002155	0.002155	
1	Flight Distance	-0.000175	0.000175	
17	Arrival Delay in Minutes	-0.006953	0.006953	
0	Age	-0.008913	0.008913	
6	Inflight wifi service	-0.095370	0.095370	
4	Food and drink	-0.197689	0.197689	
3	Departure/Arrival time convenient	-0.219083	0.219083	
20	Class_Eco	-0.729280	0.729280	
19	Type of Travel_Personal Travel	-0.739723	0.739723	
21	Class_Eco Plus	-0.787932	0.787932	
18	Customer Type_disloyal Customer	-1.878154	1.878154	

6. Interpretation of Model Coefficients and Business Recommendations

Logistic Regression coefficients indicate the change in the log-odds of the dependent variable (satisfaction) for a one-unit increase in the predictor variable, holding all other predictors constant. A positive coefficient means that as the feature increases, the log-odds of satisfaction increase, thus increasing the probability of satisfaction. A negative coefficient means the opposite.

From the sorted coefficients, we can identify the most impactful features:

Key Drivers of Satisfaction (Positive Impact):

- **Inflight entertainment (0.72):** This is the strongest positive driver. Passengers who rate inflight entertainment highly are significantly more likely to be satisfied.
- **On-board service (0.31):** Good on-board service contributes positively to satisfaction.

- **Checkin service (0.28):** Efficient check-in processes improve passenger satisfaction.
- **Seat comfort (0.28):** Comfortable seats are a notable factor in passenger happiness.
- **Ease of Online booking (0.25):** A user-friendly online booking system enhances satisfaction.
- **Leg room service (0.24):** More legroom is associated with higher satisfaction.
- **Online boarding (0.14):** A smooth online boarding experience is beneficial.
- **Gate location (0.13):** Convenient gate locations improve satisfaction.
- **Online support (0.11):** Good online support contributes to positive experiences.
- **Baggage handling (0.09):** Efficient baggage handling also plays a role.

Key Drivers of Dissatisfaction (Negative Impact):

- **Customer Type_disloyal Customer (-1.88):** Being a disloyal customer significantly decreases the probability of satisfaction. This indicates a fundamental difference in expectations or experiences compared to loyal customers.
- **Class_Eco Plus (-0.79) and Class_Eco (-0.73):** Compared to Business Class (the baseline since `drop_first=True` was used for encoding), passengers in Economy Plus and Economy classes are significantly less likely to be satisfied. This highlights class differences in service quality or expectations.
- **Type of Travel_Personal Travel (-0.74):** Passengers traveling for personal reasons are less likely to be satisfied compared to those traveling for business (the baseline).
- **Departure/Arrival time convenient (-0.22):** Inconvenient departure/arrival times negatively impact satisfaction.
- **Food and drink (-0.20):** Poor food and drink quality leads to dissatisfaction.
- **Inflight wifi service (-0.10):** A negative coefficient suggests that issues with inflight Wi-Fi service can lead to dissatisfaction.

Business Recommendations:

Based on these insights, the airline can implement the following strategies to improve customer satisfaction:

1. **Prioritize Inflight Entertainment:** This is the strongest positive driver. Invest in upgrading and diversifying inflight entertainment options across all classes. This could include newer movies, a wider selection of TV shows, games, and perhaps even live streaming options where feasible.
2. **Enhance Core Service Quality:** Focus on improving on-board service, check-in processes, and seat comfort. Training staff for exceptional service, streamlining check-in with self-service kiosks, and ensuring comfortable seating are crucial.
3. **Optimize Online Experience:** Improve the ease of online booking and online boarding. A seamless digital experience from booking to boarding can significantly reduce friction and improve overall perception.
4. **Address Disloyal Customers:** Investigate the specific pain points of disloyal customers. Are they experiencing consistent issues that loyal customers are not? Tailored loyalty programs or targeted improvements based on their feedback could help convert them.
5. **Segmented Service Improvements (Class & Travel Type):** Acknowledge that different customer segments (e.g., personal vs. business travelers, different classes) have varying expectations and experiences. While Business Class typically has higher satisfaction, consider targeted improvements for Economy and Economy Plus, especially concerning seat comfort, food and drink, and legroom, which are modifiable factors.
6. **Focus on Convenience:** Ensure departure/arrival times are as convenient as possible and clearly communicated. Minimize delays where possible, as they negatively impact satisfaction.
7. **Upgrade Food & Drink Offerings:** Review and improve the quality and variety of food and drink options, especially in classes where dissatisfaction is higher.
8. **Improve Inflight Wi-Fi:** Given its negative coefficient, ensure reliable and fast inflight Wi-Fi service. Address common complaints related to connectivity and pricing.

Limitations and Next Steps:

Limitations:

- **Correlation vs. Causation:** Logistic Regression identifies correlations, not necessarily direct causation. While improving inflight entertainment is linked to higher satisfaction, other underlying factors might be at play.
- **Linear Relationship Assumption:** Logistic Regression assumes a linear relationship between the independent variables and the log-odds of the dependent variable, which might not always hold true

in complex real-world scenarios.

- **Feature Interaction:** The current model does not explicitly account for complex interactions between features (e.g., the impact of Wi-Fi might be different for business travelers).
- **Data Bias:** The dataset represents past experiences and might not fully capture emerging trends or new passenger expectations.

Next Steps for Model Deployment:

1. **Feature Engineering & Selection:** Explore more advanced feature engineering techniques or feature selection methods to potentially improve model performance and interpretability.
2. **Advanced Models:** Experiment with more complex models like Gradient Boosting Machines (e.g., XGBoost, LightGBM) or Neural Networks, which can capture non-linear relationships and interactions, though they might be less interpretable.
3. **Threshold Optimization:** Evaluate the impact of different probability thresholds for classifying 'satisfied' vs. 'dissatisfied' to optimize for specific business goals (e.g., prioritizing recall to identify all potentially dissatisfied customers).
4. **A/B Testing:** Implement recommended changes (e.g., improved inflight entertainment) and conduct A/B tests to measure their actual impact on customer satisfaction in a controlled environment.